

Capítulo 11: Integrated Analysis

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

23 de junho de 2026

Sumário

- 1 Introdução
- 2 Integração Multi-Ômica
- 3 Caso de Estudo: Ciclo da Trealose em Levedura
- 4 Análise de Vias e Redes Integradas
- 5 Modelagem de Sistemas Integrados
- 6 Exercícios
- 7 Referências

Visão Geral do Capítulo

Objetivos de Aprendizagem

- Compreender a abordagem de biologia de sistemas
- Dominar estratégias de integração multi-ômica
- Analisar caso de estudo: ciclo da trealose em levedura
- Aplicar métodos de análise de vias integradas
- Construir modelos de sistemas integrados

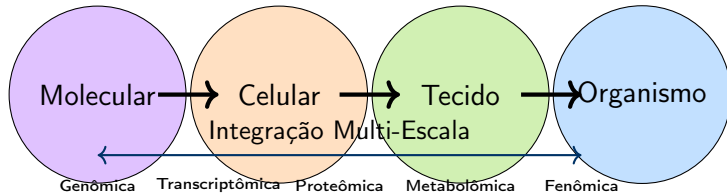
Por que Análise Integrada?

- Visão holística de sistemas biológicos complexos
- Descoberta de mecanismos regulatórios ocultos
- Predição de comportamento do sistema
- Identificação de alvos terapêuticos

Abordagem de Biologia de Sistemas

Definição: Biologia de Sistemas

Estudo de sistemas biológicos através da integração quantitativa de dados multi-ômicos para compreender propriedades emergentes e comportamento do sistema como um todo.



Estratégia de Integração Multi-Ômica

Camadas de Dados:

- **Genômica:** sequência, variações
- **Transcriptômica:** expressão gênica
- **Proteômica:** abundância de proteínas
- **Metabolômica:** concentrações de metabólitos
- **Fluxômica:** fluxos metabólicos

Desafios:

- Escalas diferentes
- Heterogeneidade de dados
- Ruído experimental
- Dados faltantes
- Causalidade vs correlação
- Complexidade computacional

Importante!

A integração efetiva requer métodos estatísticos robustos e modelagem matemática rigorosa.

Caso de Estudo: Ciclo da Trealose em Levedura

Por que Trealose?

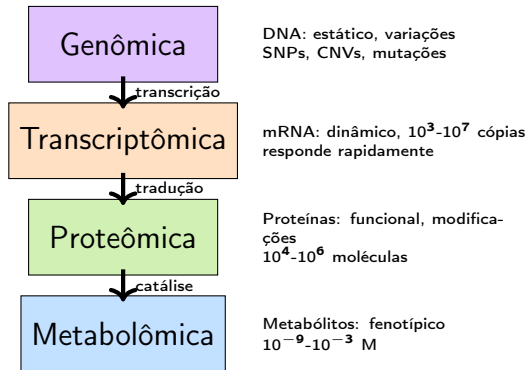
Saccharomyces cerevisiae utiliza trealose como:

- Proteção contra estresse (calor, frio, dessecação)
- Reserva de carbono
- Sinalização celular
- Regulação do metabolismo

Vantagens para Estudo Integrado

- Sistema bem caracterizado
- Via regulada em múltiplos níveis
- Dados multi-ômicos disponíveis
- Respostas mensuráveis
- Modelo para outros sistemas de estresse

Visão Geral das Camadas Ômicas



Importante!

Cada camada adiciona informação complementar sobre o estado do sistema.

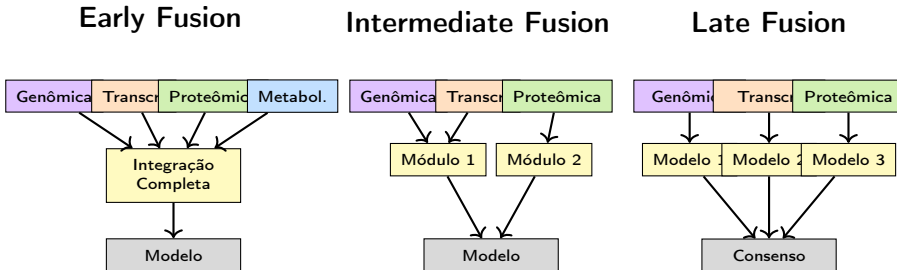
Tipos e Características de Dados

Ômica	Técnica	Elementos	Faixa Dinâmica	Ruído
Genômica	WGS, SNP	$\sim 10^4$ genes	Baixa	Baixo
Transcriptômica	RNA-seq	$\sim 10^4$ mRNAs	Alta (10^6)	Médio
Proteômica	MS, Western	$\sim 10^3$ prot.	Média (10^4)	Alto
Metabolômica	MS, NMR	$\sim 10^2$ met.	Muito alta (10^9)	Alto
Fluxômica	^{13}C -MFA	$\sim 10^2$ fluxos	Média	Médio

Implicações para Integração

- Normalização entre escalas
- Tratamento de valores faltantes
- Métodos estatísticos robustos
- Validação experimental

Estratégias de Integração: Early, Intermediate, Late Fusion



Métodos Baseados em Correlação

Análise de Correlação Multi-Ômica

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

Métodos:

- Correlação de Pearson (linear)
- Correlação de Spearman (monotônica)
- Distância de correlação
- Informação mútua
- Correlação parcial
- Correlação canônica (CCA)

Aplicações:

- mRNA-proteína
- Proteína-metabólito
- Expressão-fluxo
- Módulos co-regulados

Cuidado!

Correlação $\neq$ Causalidade

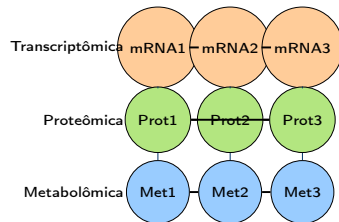
Integração Baseada em Redes

Construção de Redes

1. **Nós:** biomoléculas (genes, proteínas, metabólitos)
2. **Arestas:** interações, correlações, regulações
3. **Pesos:** força da interação

Tipos de Redes:

- Co-expressão
- Interação proteína-proteína
- Regulação gênica
- Metabólica
- Multi-camada (multiplex)



Análise:

- Centralidade
- Modularidade
- Comunidades

Abordagens de Machine Learning

Métodos Supervisionados

- **Random Forest:** classificação multi-ômica
- **SVM:** identificação de biomarcadores
- **Neural Networks:** predição de fenótipo
- **Elastic Net:** seleção de features

Métodos Não-Supervisionados

- **PCA/PLS:** redução de dimensionalidade
- **t-SNE/UMAP:** visualização
- **Clustering:** identificação de subgrupos
- **NMF:** decomposição de matrizes
- **Autoencoders:** aprendizado de representações

Exemplo: MOFA (Multi-Omics Factor Analysis)

Decomposição fatorial para identificar fontes de variação compartilhadas e específicas entre camadas ômicas.

Desafios: Heterogeneidade de Dados

Tipos de Heterogeneidade:

1. **Escala:** valores absolutos vs relativos
2. **Distribuição:** normal, log-normal, Poisson
3. **Dimensionalidade:** 10^2 a 10^5 features
4. **Esparsidade:** muitos zeros
5. **Ruído:** técnico e biológico

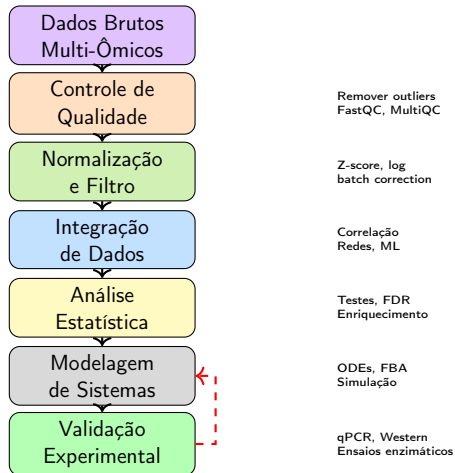
Soluções:

- Normalização apropriada
- Transformações (log, Box-Cox)
- Batch correction
- Imputação de valores faltantes
- Regularização
- Métodos robustos

Normalização Multi-Ômica

$$X_{norm} = \frac{X - \mu}{\sigma} \quad \text{ou} \quad X_{norm} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

Fluxo de Trabalho Multi-Ômico



Exemplo Python: Integração de Dados

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6
7 # Carregar dados multi-ômicos
8 transcriptomics = pd.read_csv('transcriptomics.csv', index_col=0)
9 proteomics = pd.read_csv('proteomics.csv', index_col=0)
10 metabolomics = pd.read_csv('metabolomics.csv', index_col=0)
11
12 # Normalizar cada dataset (z-score)
13 def normalize_zscore(df):
14     return (df - df.mean()) / df.std()
15
16 trans_norm = normalize_zscore(transcriptomics)
17 prot_norm = normalize_zscore(proteomics)
18 metab_norm = normalize_zscore(metabolomics)
19
20 # Correlacao mRNA-proteina
21 correlation_matrix = pd.DataFrame(
22     index=trans_norm.index,
23     columns=prot_norm.index
24 )
25
26 for gene in trans_norm.index:
27     for protein in prot_norm.index:
28         r, p = stats.pearsonr(trans_norm.loc[gene],
29                               prot_norm.loc[protein])
30         if p < 0.05: # significativo
31             correlation_matrix.loc[gene, protein] = r
32
33 # Visualizar heatmap
34 plt.figure(figsize=(12, 10))
35 sns.heatmap(correlation_matrix.astype(float),
36             cmap='RdBu_r', center=0, vmin=-1, vmax=1)
37 plt.title('Correlacao mRNA-Proteina')
```

Listing 1: Integração multi-ômica com correlação

Background Biológico: Resposta ao Estresse em *S. cerevisiae*

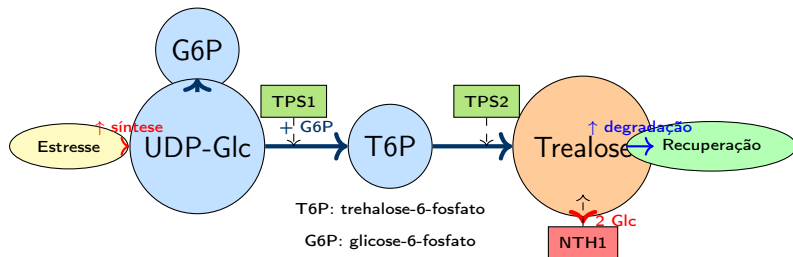
Trealose como Protetor Celular

- Dissacarídeo não-redutor (α -D-glicopiranosil- α -D-glicopiranosídeo)
- Acumula durante estresse térmico, osmótico, oxidativo
- Estabiliza membranas e proteínas
- Pode atingir 20% do peso seco celular
- Mobilizada quando condições melhoram

Relevância Industrial

- Panificação: tolerância ao estresse
- Fermentação alcoólica: robustez
- Produção de biocombustíveis
- Biotecnologia: engenharia de robustez

Via de Biossíntese e Degradação da Trealose



Importante!

Via regulada transcricionalmente, pós-traducionalmente e alostericamente.

Enzimas Chave: TPS1, TPS2, NTH1

TPS1 (Trealose-6-P Sintase)

- Catalisa: $\text{UDP-Glc} + \text{G6P} \rightarrow \text{T6P}$
- Subunidade catalítica
- Gene essencial
- Regulação: PKA, Snf1

$$v_{\text{TPS1}} = \frac{V_{\text{max}}[\text{UDP-Glc}][\text{G6P}]}{K_m^{\text{UDP}} K_m^{\text{G6P}}}$$

NTH1 (Trealase Neutra)

- Catalisa: $\text{Trealose} \rightarrow 2 \text{ Glc}$
- Mobiliza reservas
- Ativa por PKA (cAMP)
- Inibida durante estresse

$$v_{\text{NTH1}} = \frac{V_{\text{max}}[\text{Tre}]}{K_m + [\text{Tre}]} \cdot f(\text{PKA})$$

TPS2 (Trealose-6-P Fosfatase)

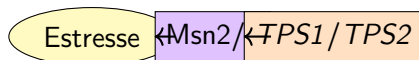
- Catalisa: $\text{T6P} \rightarrow \text{Trealose} + \text{Pi}$
- Remove T6P (tóxico em excesso)
- Essencial para crescimento

Complexo TSC

TPS1, TPS2 e subunidades regulatórias (TSL1, TPS3) formam complexo de 800 kDa

Mecanismos Regulatórios

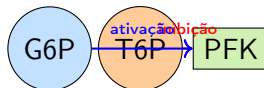
Regulação Transcricional



Regulação Pós-Traducional



Regulação Alostérica



T6P inibe glicólise quando há acúmulo de trealose

Sistema de feedback multi-nível garante homeostase de trealose e G6P.

Dados Experimentais: Multi-Ômicos

Dataset Integrado (Choque Térmico 37°C)

Tempo (min)	mRNA TPS1	Proteína TPS1	[Trealose] mM
0	1.0	1.0	5
15	3.2	1.2	8
30	8.5	2.8	25
60	12.0	5.5	85
90	10.5	7.2	120
120	6.8	6.8	130

Observações

- mRNA responde rapidamente (15 min)
- Proteína segue com atraso (30-60 min)
- Trealose acumula gradualmente
- Saturação em 2h

Construção do Modelo Matemático

Sistema de EDOs para Ciclo da Trealose

$$\begin{aligned}\frac{d[mRNA_{TPS1}]}{dt} &= k_{trans} \cdot f(stress) - \delta_{mRNA}[mRNA_{TPS1}] \\ \frac{d[TPS1]}{dt} &= k_{transl}[mRNA_{TPS1}] - \delta_{prot}[TPS1] \\ \frac{d[T6P]}{dt} &= v_{TPS1} - v_{TPS2} \\ \frac{d[Tre]}{dt} &= v_{TPS2} - v_{NTH1}\end{aligned}$$

onde:

- $f(stress) = \frac{stress^n}{K_d^n + stress^n}$ (função de Hill)
- $v_{TPS1} = \frac{V_{max}^{TPS1}[TPS1][G6P][UDP-Glc]}{(K_m^{G6P} + [G6P])(K_m^{UDP} + [UDP-Glc])}$
- $v_{NTH1} = \frac{V_{max}^{NTH1}[NTH1][Tre]}{K_m^{Tre} + [Tre]} \cdot f(PKA)$

Estimação de Parâmetros

Métodos de Estimação:

1. Mínimos Quadrados:

$$\min_{\theta} \sum_i (y_i^{obs} - y_i^{model}(\theta))^2$$

2. Maximum Likelihood:

$$\max_{\theta} \prod_i P(y_i^{obs} | \theta)$$

3. Bayesiano (MCMC):

$$P(\theta|y) \propto P(y|\theta)P(\theta)$$

Restrições:

- $V_{max} > 0$
- $K_m > 0$
- $\delta > 0$
- Limites biológicos

Ferramentas

- `scipy.optimize`
- `lmfit`
- `PyMC3`
- `COPASI`
- `MATLAB simbiology`

Importante!

Usar dados multi-ômicos para restringir espaço de parâmetros.

Validação do Modelo

Critérios de Validação:

1. **Ajuste aos dados:** R^2 , RMSE
2. **Validação cruzada:** hold-out, k-fold
3. **Predições independentes:** novas condições
4. **Análise de sensibilidade:** parâmetros críticos
5. **Testes experimentais:** mutantes, perturbações

Métricas:

- $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$
- $RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$
- $AIC = 2k - 2 \ln(L)$
- $BIC = k \ln(n) - 2 \ln(L)$

Validação Experimental

- Testar mutantes: $\Delta tps1$, $\Delta nth1$
- Perturbações químicas: inibidores de PKA
- Diferentes condições de estresse
- Comparar predições com medições

Predições e Verificação Experimental

Predições do Modelo

1. Aumento de 10x na expressão de TPS1 sob estresse leva a acúmulo 2x mais rápido de trealose
2. Deleção de NTH1 resulta em acúmulo de trealose mesmo após estresse
3. Inibição de PKA aumenta trealose basal em 3x
4. Pré-adaptação ao estresse acelera resposta em 40%

Verificação Experimental

Condição	Predição	Experimental
Superexpressão TPS1	2.1x rápido	1.9x rápido
$\Delta nth1$	130 mM final	125 mM final
Inibidor PKA	3.2x basal	3.0x basal
Pré-adaptação	40% ↓ tempo	38% ↓ tempo

Concordância entre modelo e experimentos valida abordagem integrada!

Implementação Python: Modelo de Trealose

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def trehalose_model(y, t, params):
6     """Modelo do ciclo da trealose"""
7     mRNA_TPS1, TPS1, T6P, Tre = y
8
9     k_trans, delta_mRNA, k_transl, delta_prot = params[:4]
10    Vmax_TPS1, Km_G6P, Vmax_TPS2, Km_T6P = params[4:8]
11    Vmax_NTH1, Km_Tre, stress_level = params[8:]
12
13    # Constantes
14    G6P = 5.0 # mM (assumido constante)
15    UDP_Glc = 2.0 # mM
16
17    # Regulacao transcritcional (Hill function)
18    n_hill = 4
19    Kd_stress = 0.5
20    f_stress = stress_level**n_hill / (Kd_stress**n_hill + stress_level**n_hill)
21
22    # Taxas de reacao
23    v_TPS1 = (Vmax_TPS1 * TPS1 * G6P * UDP_Glc) / ((Km_G6P + G6P) * (1.0 + UDP_Glc))
24    v_TPS2 = (Vmax_TPS2 * T6P) / (Km_T6P + T6P)
25    v_NTH1 = (Vmax_NTH1 * Tre) / (Km_Tre + Tre)
26
27    # EDOs
28    d_mRNA = k_trans * f_stress - delta_mRNA * mRNA_TPS1
29    d_TPS1 = k_transl * mRNA_TPS1 - delta_prot * TPS1
30    d_T6P = v_TPS1 - v_TPS2
31    d_Tre = v_TPS2 - v_NTH1
32
33    return [d_mRNA, d_TPS1, d_T6P, d_Tre]
```

Listing 2: Simulação do ciclo da trealose

Simulação e Ajuste aos Dados

```
1 # Parametros (estimados)
2 params = [
3     5.0,    # k_trans
4     0.1,    # delta_mRNA
5     2.0,    # k_transl
6     0.05,   # delta_prot
7     10.0,   # Vmax_TPS1
8     0.5,    # Km_G6P
9     15.0,   # Vmax_TPS2
10    0.2,    # Km_T6P
11    8.0,    # Vmax_NTH1
12    10.0,   # Km_Tre
13    1.0     # stress_level (heat shock)
14 ]
15
16 # Condições iniciais
17 y0 = [1.0, 1.0, 0.1, 5.0] # mRNA, TPS1, T6P, Tre
18
19 # Tempo de simulação
20 t = np.linspace(0, 120, 100) # 0 a 120 min
21
22 # Resolver EDOs
23 sol = odeint(trehalose_model, y0, t, args=(params,))
24
25 # Plotar resultados
26 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
27 axes[0,0].plot(t, sol[:,0], 'b-', label='mRNA TPS1')
28 axes[0,1].plot(t, sol[:,1], 'g-', label='Proteína TPS1')
29 axes[1,0].plot(t, sol[:,2], 'r-', label='T6P')
30 axes[1,1].plot(t, sol[:,3], 'orange', label='Trehalose')
31
32 # Adicionar dados experimentais
33 t_exp = [0, 15, 30, 60, 90, 120]
34 tre_exp = [5, 8, 25, 85, 120, 130]
35 axes[1,1].plot(t_exp, tre_exp, 'ko', label='Experimental')
```

Listing 3: Ajuste do modelo aos dados experimentais

Análise de Enriquecimento de Vias (Pathway Enrichment)

Definição: Análise de Enriquecimento

Método estatístico para identificar se um conjunto de genes/proteínas está super-representado em vias biológicas ou processos específicos.

Teste de Hipergeométrico

$$p = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

onde:

- N = total de genes no genoma
- K = genes na via de interesse
- n = genes no conjunto (ex: diferencialmente expressos)
- k = genes no conjunto que estão na via

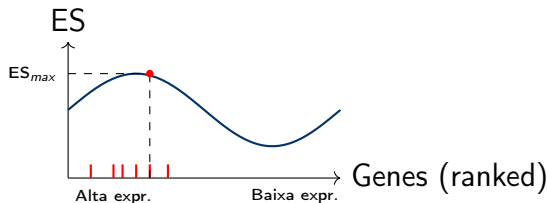
Ferramentas

DAVID, Enrichr, g:Profiler, clusterProfiler, GSEA

Gene Set Enrichment Analysis (GSEA)

Vantagem sobre ORA:

- Usa ranking completo de genes
- Não requer threshold arbitrário
- Detecta mudanças coordenadas sutis
- Considera magnitude das mudanças



Procedimento:

1. Rankear genes (fold-change, t-statistic)
2. Calcular Enrichment Score (ES)
3. Permutar para p-valor
4. Corrigir para múltiplos testes (FDR)

Interpretação:

- ES positivo: via ativada
- ES negativo: via reprimida
- Magnitude: força do enriquecimento

Reconstrução de Redes a partir de Multi-Ômicos

Abordagens de Reconstrução

1. **Correlação:** Pearson, Spearman
 - Simples, rápido
 - Não distingue correlação direta vs indireta
2. **Correlação Parcial:** controla co-variáveis

$$\rho_{XY|Z} = \frac{\rho_{XY} - \rho_{XZ}\rho_{YZ}}{\sqrt{(1 - \rho_{XZ}^2)(1 - \rho_{YZ}^2)}}$$

3. **Informação Mútua:**

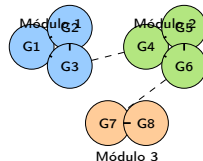
$$I(X; Y) = \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

4. **Graphical LASSO:** inferência de grafos Gaussianos
5. **Redes Bayesianas:** aprendem causalidade

Detecção de Módulos e Clustering

Métodos de Clustering:

- **K-means:** clusters esféricos
- **Hierárquico:** dendrograma
- **DBSCAN:** baseado em densidade
- **Louvain:** modularidade em grafos
- **Spectral:** autovalores da matriz de adjacência



Modularidade

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Aplicações:

- Identificar processos biológicos
- Co-regulação
- Predição de função

Exemplo: Módulo de Resposta ao Estresse

Genes co-expressos: HSP, TPS, CTT, SSA - todos regulados por Msn2/4

Identificação de Reguladores e Hubs

Métricas de Centralidade

- **Grau (Degree):** número de conexões

$$k_i = \sum_j A_{ij}$$

- **Betweenness:** fração de caminhos mais curtos que passam pelo nó

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

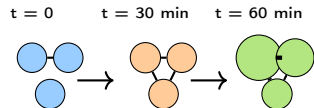
- **Closeness:** proximidade média a outros nós

$$C_C(i) = \frac{n-1}{\sum_j d(i,j)}$$

- **Eigenvector:** conectado a nós importantes
- **PageRank:** variante de eigenvector centralidade

Redes que Mudam no Tempo:

- Topologia varia com condições
- Arestas aparecem/desaparecem
- Pesos mudam
- Módulos se reorganizam



Resposta ao
estresse térmico

Métodos:

- Time-lagged correlation
- Dynamic Bayesian Networks
- Graphical Granger causality
- Sliding window analysis

Importante!

Redes dinâmicas revelam sequência temporal de eventos regulatórios.

Exemplo Python: Análise de Redes com NetworkX

```
1 import networkx as nx
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import pearsonr
5
6 # Construir rede de correlacao
7 def build_correlation_network(data, threshold=0.7):
8     """Cria rede baseada em correlacoes"""
9     G = nx.Graph()
10
11     # Adicionar nos
12     for gene in data.index:
13         G.add_node(gene)
14
15     # Adicionar arestas (correlacoes significativas)
16     for i, gene1 in enumerate(data.index):
17         for gene2 in data.index[i+1:]:
18             r, p = pearsonr(data.loc[gene1], data.loc[gene2])
19             if abs(r) > threshold and p < 0.05:
20                 G.add_edge(gene1, gene2, weight=abs(r))
21
22     return G
23
24 # Analisar centralidade
25 def analyze_network(G):
26     """Calcula metricas de centralidade"""
27     degree_cent = nx.degree_centrality(G)
28     between_cent = nx.betweenness_centrality(G)
29     close_cent = nx.closeness_centrality(G)
30
31     # Identificar hubs (top 10% degree)
32     threshold = sorted(degree_cent.values())[-len(G)//10]
33     hubs = [node for node, cent in degree_cent.items()
34             if cent >= threshold]
35
36     return degree_cent, between_cent, close_cent, hubs
```

Listing 4: Construção e análise de rede multi-ômica

Análise de Enriquecimento com Python

```
1 import gseapy as gp
2 import pandas as pd
3
4 # Dados de expressao diferencial
5 de_genes = pd.read_csv('diff_expr.csv')
6 de_genes = de_genes.sort_values('log2FC', ascending=False)
7
8 # Criar ranking
9 gene_rank = de_genes.set_index('gene')['log2FC'].to_dict()
10
11 # Executar GSEA pre-ranked
12 gsea_results = gp.prerank(
13     rnk=gene_rank,
14     gene_sets='KEGG_2021_Human', # banco de vias
15     processes=4,
16     permutation_num=1000,
17     outdir='gsea_output',
18     format='png',
19     seed=42
20 )
21
22 # Vias significativamente enriquecidas
23 enriched_pathways = gsea_results.res2d[
24     gsea_results.res2d['FDR q-val'] < 0.05
25 ]
26
27 print(f"Vias enriquecidas: {len(enriched_pathways)}")
28 print(enriched_pathways[['Term', 'ES', 'NES', 'FDR q-val']].head(10))
29
30 # Over-representation analysis (ORA)
31 enrich_ora = gp.enrichr(
32     gene_list=de_genes[de_genes['padj'] < 0.05]['gene'].tolist(),
33     gene_sets='GO_Biological_Process_2021',
34     organism='Yeast',
35     outdir='enrichr_output'
36 )
```

Listing 5: GSEA com gseapy

Modelos de Escala Genômica com Restrições Ômicas

Definição: Genome-Scale Model (GEM)

Reconstrução matemática do metabolismo completo de um organismo, incluindo todas as reações conhecidas, genes e estequiometria.

Integração de Dados de Expressão

- **E-Flux**: usar níveis de expressão como proxy para capacidades enzimáticas

$$v_{max,i} \propto \text{expression}_i$$

- **GIMME**: minimizar diferenças entre fluxos e expressão

$$\min \sum_i |v_i - v_i^{\text{target}}| \quad \text{s.t.} \quad Sv = 0$$

- **iMAT**: maximizar consistência com dados binários (on/off)

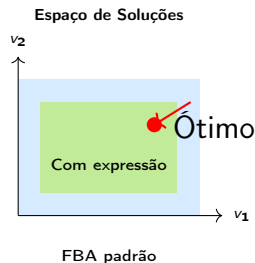
Constraint-Based Modeling com Transcriptômica

Análise de Balanço de Fluxos (FBA):

$$\begin{aligned} \max_v \quad & c^T v \\ \text{s.t.} \quad & Sv = 0 \\ & lb \leq v \leq ub \end{aligned}$$

Com dados de expressão:

- Ajustar lb e ub baseado em níveis de mRNA
- Genes altamente expressos: ub alto
- Genes não-expressos: $v = 0$
- Genes intermediários: ub proporcional



Importante!

Dados de expressão restringem espaço de soluções, melhorando previsões.

Modelos Cinéticos com Proteômica/Metabolômica

Integração Multi-Nível

- **Proteômica:** concentrações enzimáticas ($[E]$)
- **Metabolômica:** concentrações de substratos/produtos ($[S]$, $[P]$)
- **Fluxômica:** fluxos metabólicos (v)

Modelo Cinético Integrado

Para cada reação i :

$$v_i = [E_i] \cdot k_{cat,i} \cdot \frac{[S_i]}{K_m^S + [S_i]} \cdot \left(1 - \frac{[P_i]}{[S_i]K_{eq,i}}\right)$$

Sistema de EDOs:

$$\frac{d[S_i]}{dt} = \sum_j s_{ij} v_j$$

onde s_{ij} é o coeficiente estequiométrico.

Dados proteômicos e metabolômicos fornecem valores absolutos para parametrização!

Abordagens de Modelagem Híbrida

Combinando Diferentes Formalismos:

1. FBA + ODEs:

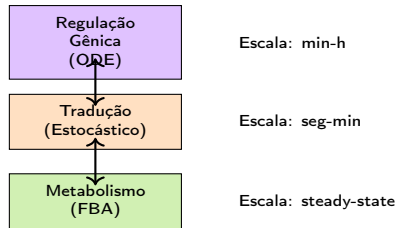
- FBA para metabolismo central
- ODEs para regulação

2. Estocástico + Determinístico:

- Gillespie para expressão gênica
- ODEs para metabolismo

3. Multi-escala:

- Molecular (ms)
- Celular (min-h)
- População (h-dias)



Vantagens

- Captura múltiplas escalas
- Usa melhor formalismo para cada nível
- Mais realista

Validação com Dados Independentes

Estratégias de Validação

1. **Hold-out:** treinar em 70%, testar em 30%
2. **Validação cruzada:** k-fold ($k=5$ ou 10)
3. **Leave-one-out:** para datasets pequenos
4. **Validação temporal:** treinar em tempo t , validar em $t + \Delta t$
5. **Validação experimental:** perturbações independentes

Tipos de Dados para Validação:

- Mutantes deletados
- Superexpressões
- Diferentes condições nutricionais
- Resposta a drogas
- Dados de literatura

Métricas de Performance:

- Acurácia de predição
- Correlação observado vs predito
- Erro médio absoluto
- Sensibilidade/especificidade
- AUC-ROC

Exemplo Python: FBA Integrado com Transcriptômica

```
1 import cobra
2 import pandas as pd
3
4 # Carregar modelo genome-scale
5 model = cobra.io.read_sbml_model('iMM904.xml') # S. cerevisiae
6
7 # Carregar dados de expressao (RNA-seq)
8 expression_data = pd.read_csv('expression_stress.csv', index_col=0)
9 expression_data = expression_data / expression_data.max() # normalizar 0-1
10
11 # Integrar dados de expressao (metodo E-Flux)
12 def integrate_expression(model, expression, percentile=25):
13     """Ajusta bounds baseado em expressao"""
14     for reaction in model.reactions:
15         # Encontrar genes associados
16         genes = reaction.genes
17
18         if len(genes) > 0:
19             # Media de expressao dos genes
20             gene_expr = [expression.get(g.id, 0.5) for g in genes]
21             avg_expr = sum(gene_expr) / len(gene_expr)
22
23             # Ajustar upper bound
24             if avg_expr < percentile/100:
25                 # Gene pouco expresso: bloquear reacao
26                 reaction.upper_bound = 0
27                 reaction.lower_bound = 0
28             else:
29                 # Escalar bound pela expressao
30                 original_ub = reaction.upper_bound
31                 if original_ub > 0:
32                     reaction.upper_bound = original_ub * avg_expr
33
34 # Aplicar restricoes de expressao
35 integrate_expression(model, expression_data)
36
37 # Executar FBA
38 solution = model.optimize()
39 print(f"Taxa de crescimento predita: {solution.objective_value:.3f}")
```

Listing 6: FBA com restrições de expressão usando COBRApy

Modelo Híbrido: Regulação + Metabolismo

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import cobra
4
5 def hybrid_model(y, t, model, params):
6     """Modelo híbrido: ODE para regulação + FBA para metabolismo"""
7     mRNA, protein = y
8     k_trans, k_transl, delta_mRNA, delta_prot = params
9
10    # Parte 1: Regulação genica (ODE)
11    stress = 1.0 if t < 60 else 0.0 # estresse nos primeiros 60 min
12
13    dmRNA_dt = k_trans * stress - delta_mRNA * mRNA
14    dprotein_dt = k_transl * mRNA - delta_prot * protein
15
16    # Parte 2: Ajustar metabolismo baseado em nível proteico
17    # Exemplo: protein = TPS1, afeta fluxo de trealose
18    for rxn in model.reactions:
19        if 'TPS1' in rxn.id:
20            # Escalar capacidade pela abundancia proteica
21            rxn.upper_bound = 10.0 * protein
22
23    # Resolver FBA
24    try:
25        solution = model.optimize()
26        growth_rate = solution.objective_value
27        trehalose_flux = solution.fluxes.get('TPS1-rxn', 0.0)
28    except:
29        growth_rate = 0.0
30        trehalose_flux = 0.0
31
32    # Feedback: trealose afeta transcrição
33    feedback = trehalose_flux / 10.0
34    dmRNA_dt += feedback
35
36    return [dmRNA_dt, dprotein_dt]
37
38 # Condições iniciais e parametros
39 y0 = [1.0, 1.0]
40 params = [5.0, 2.0, 0.1, 0.05]
41 t = np.linspace(0, 120, 100)
42
43 # Simular
44 sol = odeint(hybrid_model, y0, t, args=(model, params))
```

Listing 7: Modelo híbrido simples

Exercícios - Parte 1: Correlação Multi-Ômica

Questão 1: Análise de Correlação

Dados de transcriptômica e proteômica para 5 genes sob condição de estresse:

Gene	$\log_2(\text{mRNA})$	$\log_2(\text{Proteína})$
<i>TPS1</i>	3.2	1.8
<i>HSP104</i>	4.5	2.1
<i>CTT1</i>	2.8	1.5
<i>SSA1</i>	3.7	2.0
<i>GLK1</i>	0.5	0.3

1. Calcule a correlação de Pearson entre mRNA e proteína
2. É estatisticamente significativa?
3. Interprete biologicamente o resultado

Exercícios - Parte 2: Enriquecimento de Vias

Questão 2: Teste de Hipergeométrico

Em um experimento de estresse oxidativo, 150 genes foram diferencialmente expressos de um total de 6000 genes no genoma. A via de "Resposta ao Estresse Oxidativo" contém 80 genes, e 25 deles estão no conjunto de genes diferencialmente expressos.

1. Calcule o p-valor usando o teste hipergeométrico
2. A via está significativamente enriquecida ($\alpha = 0.05$)?
3. Se aplicarmos correção de Bonferroni para 300 vias testadas, ainda é significativa?

Questão 3: GSEA

1. Explique a diferença entre ORA e GSEA
2. Por que GSEA pode detectar mudanças sutis que ORA perde?
3. Quando você usaria cada método?

Exercícios - Parte 3: Construção de Modelos

Questão 4: Projeto - Modelagem Integrada

Construa um modelo integrado para a via de glicólise em *S. cerevisiae*:

1. Baixe dados de RNA-seq, proteômica e metabolômica do GEO
2. Identifique genes/proteínas/metabólitos da glicólise
3. Construa modelo cinético simplificado (3-5 reações chave)
4. Estime parâmetros usando dados experimentais
5. Valide o modelo com dados de mutantes
6. Simule resposta a mudança de glicose extracelular

Entregáveis:

- Código Python documentado
- Gráficos de ajuste e validação
- Análise de sensibilidade de parâmetros
- Relatório interpretando resultados biologicamente

Exercícios - Parte 4: Análise de Redes

Questão 5: Construção de Rede de Co-expressão

Usando dados de RNA-seq de séries temporais (0, 15, 30, 60, 120 min após estresse):

1. Calcule matriz de correlação entre todos os pares de genes
2. Construa rede mantendo correlações $|r| > 0.8$ e $p < 0.01$
3. Identifique módulos usando algoritmo de Louvain
4. Calcule centralidade de grau, betweenness e closeness
5. Identifique top 10 hubs
6. Faça análise de enriquecimento GO para cada módulo

Questão 6: FBA com Restrições

1. Carregue modelo genome-scale de *S. cerevisiae* (iMM904)
2. Integre dados de expressão usando método E-Flux
3. Compare taxa de crescimento predita com/sem restrições
4. Identifique reações essenciais sob estresse

Referências Principais

-  Klipp E. et al. (2016). *Systems Biology: A Textbook*, 2nd ed. Wiley-VCH.
-  Palsson B.O. (2015). *Systems Biology: Constraint-based Reconstruction and Analysis*. Cambridge University Press.
-  Ideker T., Galitski T., Hood L. (2001). A new approach to decoding life: systems biology. *Ann Rev Genomics Hum Genet*, 2:343-372.
-  Hasin Y. et al. (2017). Multi-omics approaches to disease. *Genome Biology*, 18:83.
-  Subramanian A. et al. (2005). Gene set enrichment analysis: a knowledge-based approach. *PNAS*, 102(43):15545-15550.

Reviews Multi-Ômicos

- Ritchie M.D. et al. (2015). Methods of integrating data to uncover genotype-phenotype interactions. *Nat Rev Genet*.
- Huang S. et al. (2017). More is better: recent progress in multi-omics data integration methods. *Front Genet*.
- Cavill R. et al. (2016). Transcriptomic and metabolomic data integration. *Brief Bioinform*.

Bancos de Dados e Ferramentas

- **COBRApy**: <https://opencobra.github.io/cobrapy/>
- **NetworkX**: <https://networkx.org>
- **GSEAPy**: <https://gseapy.readthedocs.io>
- **Enrichr**: <https://maayanlab.cloud/Enrichr/>
- **STRING**: <https://string-db.org>
- **KEGG**: <https://www.genome.jp/kegg/>
- **Reactome**: <https://reactome.org>

Tutoriais e Cursos

- **Systems Biology Course (MIT):** <https://ocw.mit.edu>
- **Network Analysis in Systems Biology:** <https://www.coursera.org>
- **Multi-omics Integration:** <https://www.ebi.ac.uk/training>

Pacotes Python/R

- **Python:** cobra, networkx, gseapy, scanpy, scipy, scikit-learn
- **R:** clusterProfiler, WGCNA, mixOmics, limma, DESeq2

Livros Especializados

- Alon U. (2019). *An Introduction to Systems Biology*, 2nd ed.
- Voit E.O. (2017). *A First Course in Systems Biology*, 2nd ed.
- Covert M.W. (2014). *Fundamentals of Systems Biology*.

Obrigado!

Capítulo 11: Integrated Analysis

Tópicos Abordados:

- Integração multi-ômica
- Caso de estudo: ciclo da trealose
- Análise de vias e redes
- Modelagem de sistemas integrados

Dúvidas? Perguntas?